CloudM Migrate — M365 → Google Migration Execution

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Outcome . . . 4

Approach . . . 4

Why it is low-risk . . . 4

Solution Overview 5

What this solution is . . . 5

How it works . . . 5

Reference model — naming convention . . . 5

Where CloudM Migrate fits vs. native and other tools . . . 6

The wave approach . . . 6

Benefits . . . 6

Limitations & boundaries . . . 7

Deployment model . . . 7

Requirements 7

R1. CloudM Migrate platform provisioned (hosted or self-hosted) . . . 8

R2. Microsoft 365 source app registration (Entra ID) . . . 8

R3. Google Workspace destination service account with domain-wide delegation . . . 8

R4. Destination users and Shared Drives provisioned . . . 9

R5. Source→destination mapping (user list CSV) . . . 9

R6. Migration profile design decided . . . 10

R7. Change window, cutover plan, and rollback . . . 10

Global Requirements — Google Workspace & Gemini Solutions 10

1. Google Workspace / Cloud Identity tenant . . . 11

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 11

3. DNS control for the routed domains . . . 11

4. Identity & provisioning source of truth . . . 11

5. A pilot scope (OU / group) and change control . . . 12

6. Security, compliance & governance stakeholders . . . 12

Implementation 12

Stage 1 — Assess (read-only discovery) . . . 13

Stage 2 — Design . . . 13

Stage 3 — Configure . . . 13

Stage 4 — Pilot (`Migration Wave – Pilot – Wave0`) . . . 14

Stage 5 — Validate . . . 14

Stage 6 — Expand (production waves) . . . 14

Testing & Validation 15
T1. Connection authentication . . . 15
T2. Scan baseline captured . . . 15
T3. Item-count reconciliation (mail + files) . . . 15
T4. Drive & Shared Drive permissions . . . 16
T5. Mail flow after cutover . . . 16
T6. Calendar & contacts fidelity . . . 16
T7. Error triage completeness . . . 16
T8. User acceptance (pilot) . . . 17
Troubleshooting 17
Connections & authentication . . . 17
Throttling & API limits . . . 18
Item counts & delta issues . . . 18
Permissions & sharing . . . 19
Mail flow after cutover . . . 19
Destination accounts . . . 19
Best Practices 19
Sequencing & waves . . . 20
Throttling & throughput . . . 20
Validation & data integrity . . . 20
Change control & safety . . . 20
Security & credentials . . . 21
Operations . . . 21
Deliverables Summary 21
Documentation . . . 21
Python tools (5) . . . 22
Packaging . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary

Outcome

Microsoft 365 mail, files, and collaboration data are migrated to Google Workspace **reliably and with minimal user disruption**, using **CloudM Migrate** as the migration engine. Each cohort of users is moved, validated against an explicit set of gates, and cut over on a controlled schedule — so the organization completes its M365→Google migration at scale without the surprises of an unplanned cutover.

Approach

The solution is an **execution playbook**, not a tool manual. It follows the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**:

- **Configure** the CloudM platform (hosted or self-hosted) and authenticated connections to M365 (Entra app registration, source) and Google Workspace (service account with domain-wide delegation, destination).
- **Build** a reusable migration profile and a source→destination user list.
- **Run** the proven pass sequence — Scan (discover), Pre-stage (bulk historical copy), Full/Delta (incremental), Cutover (mail-routing switch), post-cutover Delta — **wave by wave**, department by department.
- **Validate** every wave against pass/fail gates (item-count reconciliation, permissions, mail flow) before it is closed.

Five Python tools automate the safe, repeatable parts — readiness assessment, user-list generation, wave/throttling planning, post-migration validation reporting, and health monitoring — all read-only or dry-run by default, with no credentials in code.

Why it is low-risk

- **Pilot-first with hard gates.** Nothing goes to production until a pilot wave passes end-to-end validation. A failed wave is contained to its cohort.
- **Pre-staging shrinks the cutover.** The bulk of data moves days ahead of the change window, so the final switch is small, fast, and reversible.
- **Documented rollback on every high-impact change.** Mail-routing/DNS cutover runs under change control with a pre-agreed revert to M365; the source stays authoritative until validation passes.
- **Vendor boundaries respected.** Google and Microsoft/Entra steps are exact and current; CloudM console internals, licensing, and fidelity specifics are deferred to CloudM's own documentation, so the playbook stays accurate as the tool evolves.

Bottom line: M365→Google migrations completed reliably on CloudM — platform connected, profiles built, pre-stage and delta passes run and validated — with minimal disruption at enterprise scale, at a

fraction of full-consulting cost.

Solution Overview

What this solution is

This solution is the **execution playbook for migrating Microsoft 365 to Google Workspace using CloudM Migrate** at enterprise scale. It covers the end-to-end procedure — standing up the CloudM platform (hosted or self-hosted), building the M365 and Google connections, designing the migration profile and user list, and running the scan / pre-stage / full / delta sequence with throttling and validation — so that mail, files, and collaboration data move reliably with minimal user disruption.

CloudM Migrate is a **third-party migration engine**. This document treats it at the **procedural playbook level**: architecture, prerequisites, connection/endpoint setup, profile and mapping design, wave and pre-stage/delta sequencing, throttling, and validation. **Deep CloudM console internals, exact screen labels, licensing tiers, and product support are deferred to CloudM's own documentation** — where a step depends on a specific CloudM version or a preview behavior, it is flagged **"verify for your version."** The Google Admin console / Cloud Identity side and the Microsoft 365 / Entra side are authored at exact detail because they are stable, publicly documented platforms.

How it works

CloudM Migrate reads from a **source** (Microsoft 365 — Exchange Online mailboxes, OneDrive, and SharePoint) and writes to a **destination** (Google Workspace — Gmail, Google Drive, Shared Drives) over each vendor's APIs. The migration is driven by four moving parts:

1. **Platform** — the CloudM Migrate engine, either **hosted** (CloudM-run SaaS) or **self-hosted**

(installed on your own Windows migration server[s], scaled out for throughput). The engine holds the migration configuration and executes the passes.

1. **Connections** — authenticated links to each ecosystem: a **Microsoft 365 / Entra app

registration **(source read) and a** Google Workspace service account / OAuth** with domain-wide delegation (destination write). These are the security identities the engine acts as.

1. **Migration profile** — the reusable configuration object that says *what* to migrate (mail,

calendar, contacts, Drive/OneDrive, SharePoint), *how* to map source users to destination users, filtering (date ranges, folder exclusions), and throttling limits.

1. **User list** — the CSV of source→destination mappings that scopes each run to a set of users

(and, for files, sites/paths → Shared Drives).

The engine runs in **passes**: a **Scan** (read-only discovery of item counts and sizes), an optional **Pre-stage** (bulk-copy the historical bulk of data ahead of cutover while users still work in M365), a **Full** pass, and repeated **Delta** passes (copy only what changed since the last pass) leading up to **cutover** (MX/mail-routing switch to Google). A final delta after cutover sweeps any last items.

Reference model — naming convention

Every artifact this solution creates is named with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in later steps, testing, troubleshooting, and the scripts:

Artifact	Example name	What it is
CloudM platform instance	`Migration Platform – Enterprise – Prod`	The hosted/self-hosted engine
M365 source connection	`Source Connection – M365 – Prod`	Entra app registration used as source
Google destination connection	`Destination Connection – Google – Prod`	GCP service account w/ DWD
Migration profile	`Migration Profile – Mail+Drive – Standard`	Reusable what/how config
Pilot wave	`Migration Wave – Pilot – Wave0`	First validated cohort (25–50 users)
Production wave	`Migration Wave – Finance – Wave3`	A departmental cohort
Pre-stage run	`Pre-stage Run – Finance – Wave3`	Bulk historical copy pre-cutover
Delta run	`Delta Run – Finance – Wave3`	Incremental changed-items copy
Validation report	`Validation Report – Finance – Wave3`	Post-migration item/permission check

Where CloudM Migrate fits vs. native and other tools

- **Native / Google-first:** Google's own **Data Migration Service** (mail only, small scale) and

GWMME / Google Workspace Migrate handle certain M365→Google moves. CloudM Migrate is chosen for **breadth** (mail + Drive + SharePoint + collaboration in one engine), **scale** (parallelism, self-hosted throughput), and **operational tooling** (per-user reporting, retriable deltas).

- **Other third-party engines:** BitTitan MigrationWiz and Google Workspace Migrate solve overlapping

problems; the engine choice is an architecture decision made before this playbook runs. This solution assumes CloudM Migrate has been selected.

The wave approach

Migration is executed **wave by wave**, never fleet-wide in one shot. Each wave is a cohort of users (typically aligned to a department or site) that flows through **pre-stage** → **delta** → **cutover** → **post-cutover delta** → **validate**. The **pilot wave** (`Migration Wave – Pilot – Wave0`) proves the profile and connections end-to-end before any production wave runs.

Benefits

- **Reliable, repeatable cutovers** — pre-staging historical data moves the bulk of bytes off the

cutover critical path; deltas keep the final switch small and fast.

• **Minimal disruption** — users keep working in M365 until their wave cuts over; only the small final delta happens during the change window.

• **Auditable** — every wave produces a validation report (item counts, permissions, mail-flow) with explicit pass/fail gates.

• **Scoped risk** — pilot-first with documented rollback; a bad wave is contained to its cohort.

Limitations & boundaries

• **Fidelity is engine- and API-bound.** Some item types migrate with reduced fidelity (e.g. certain Exchange rules, SharePoint web parts, complex OneNote); confirm the current fidelity matrix in CloudM's documentation for your version.

• **This is not a CloudM console manual.** Screen-by-screen console navigation, licensing, and entitlement are CloudM's to document; **verify for your version**.

• **Throttling is provider-imposed.** Microsoft Graph / Exchange and Google API quotas cap throughput; this solution gives sequencing and throttling *guidance*, not guaranteed rates.

• **Coexistence (free/busy, mail routing during migration)** is adjacent and may need its own design; this solution focuses on data movement and cutover.

Deployment model

This solution follows the house model: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback for every high-impact production change (mail flow, DNS/MX, sharing, cutover). No production wave proceeds until the pilot wave passes its validation gate.

Requirements

These are the **solution-specific** prerequisites for executing an M365→Google migration with CloudM Migrate. They build on the shared **[Global Requirements — Google Workspace & Gemini Solutions](global-requirements-google.md)**, which cover the baseline every Google-category solution assumes: a provisioned **Google Workspace / Cloud Identity tenant with super-admin** (GR-1), the **source Microsoft 365 / Entra environment with admin access** (GR-2), **DNS control** for the routed domains (GR-3), an **identity source of truth and provisioning model** (GR-4), a **pilot OU/group and change control** (GR-5), and **security/compliance stakeholders** (GR-6). Verify those first; the prerequisites below are additive and CloudM/migration-specific.

> ***Vendor boundary reminder.*** *CloudM platform capabilities, licensing, and console internals are deferred to CloudM's own documentation and flagged* ***"verify for your version."*** *The Google and Microsoft/Entra items below are authored at exact detail from the vendors' public docs.*

R1. CloudM Migrate platform provisioned (hosted or self-hosted)

- **Description:** A licensed CloudM Migrate environment. Either **hosted** (CloudM-run SaaS) or **self-hosted** on one or more Windows migration servers sized for the workload. Name the instance `Migration Platform – Enterprise – Prod`.
- **Why required:** The platform is the engine that holds the migration configuration and executes scan/pre-stage/delta passes. Nothing runs without it.
- **How to verify:** Sign in to the CloudM Migrate platform and confirm you can reach the migration configuration area and that your license covers M365→Google mail + files (and SharePoint if in scope). **Verify edition, license counts, and feature coverage in CloudM's licensing docs — deferred to CloudM.**
- **How to implement if missing:** Procure the license and stand up the platform per CloudM's deployment guide. For self-hosted, provision the migration server(s) meeting CloudM's stated OS, CPU/RAM, and .NET/runtime prerequisites (**verify current sizing and prerequisites for your version**), on a network with outbound HTTPS to both Microsoft and Google API endpoints.

R2. Microsoft 365 source app registration (Entra ID)

- **Description:** An **Entra ID app registration** that CloudM authenticates as to read the source: Exchange Online mailboxes, OneDrive, and SharePoint. Name it `Source Connection – M365 – Prod`.
- **Why required:** CloudM reads source data over Microsoft Graph and Exchange APIs; a properly consented app registration (or the app-based access CloudM's connector documents) is the source identity.
- **How to verify:** In the **Entra admin center** → **App registrations**, confirm the app exists, has the **application permissions** CloudM's M365 connector requires (typically Graph mail/files/sites read plus Exchange access), and that **admin consent** has been granted (green check under **API permissions**). Confirm the client secret/certificate is current and stored securely. **The exact permission scopes are defined by CloudM's connector docs — verify for your version.**
- **How to implement if missing:** Create the app registration (**Entra admin center → App registrations → New registration**), add the API permissions CloudM specifies,** Grant admin consent for the tenant**, and create a client secret or upload a certificate. Record the Application (client) ID, Directory (tenant) ID, and secret/cert into the CloudM connection — never into source-controlled files.

R3. Google Workspace destination service account with domain-wide delegation

- **Description:** A **Google Cloud service account** with **domain-wide delegation (DWD)** authorized in the Google Admin console for the OAuth scopes CloudM's Google connector requires (Gmail insert, Drive, Calendar, Contacts, Directory read). Name it `Destination Connection – Google – Prod`.

• **Why required:** CloudM writes migrated data into each destination user's Gmail/Drive by impersonating them; DWD is the mechanism that authorizes that impersonation tenant-wide.

• **How to verify:** In **Google Cloud console** → **IAM & Admin** → **Service Accounts**, confirm the account and its key exist and DWD is enabled. In **admin.google.com** → **Security** → **Access and data control** → **API controls** → **Domain-wide delegation**, confirm the service account's **client ID** is present with the required scopes listed. **Confirm the exact scope list against CloudM's Google connector docs — verify for your version.**

• **How to implement if missing:** Create the service account and a JSON key in Google Cloud; enable DWD; then in the Admin console add the service account **Client ID** under **Domain-wide delegation** with the CloudM-specified OAuth scopes (comma-separated). Also enable the required **APIs** (Gmail, Drive, Admin SDK, Calendar, People) in the Cloud project. Store the JSON key in the CloudM platform's secured credential store — never in the repo.

R4. Destination users and Shared Drives provisioned

• **Description:** The **destination Google Workspace accounts** for the wave's users exist and are licensed, and any **Shared Drives** that will receive SharePoint/team content are created and named (`<Team> Shared Drive`). Provisioning flows from the identity source of truth (GR-4) via GCDS/SCIM.

• **Why required:** CloudM writes into existing destination mailboxes/Drives; missing or unlicensed target accounts cause per-user failures.

• **How to verify:** In **admin.google.com** → **Directory** → **Users**, confirm the wave's target accounts exist and are licensed. In **Shared drives**, confirm the target Shared Drives exist.

• **How to implement if missing:** Provision users via GCDS/SCIM from the source of truth; assign licenses; pre-create Shared Drives for team/SharePoint destinations.

R5. Source→destination mapping (user list CSV)

• **Description:** A **user-list CSV** mapping each source M365 identity (UPN/primary SMTP) to its destination Google address, plus any file source→Shared Drive mappings. This is the artifact CloudM consumes to scope a run. (The included `build-migration-userlist.py` tool generates a connector-agnostic version of this from the Google directory + a source export — **align the final column headers to CloudM's import template for your version.**)

• **Why required:** CloudM scopes each scan/pre-stage/delta pass to the users in the list; an incorrect mapping migrates data to the wrong account.

• **How to verify:** Confirm every source UPN resolves to exactly one destination address that exists in the Google directory, and that there are no duplicates or unmapped rows.

• **How to implement if missing:** Export the source user list from M365/Entra, reconcile against the

Google directory, and produce the mapping CSV (the tool assists). Review with the identity owner before import.

R6. Migration profile design decided

• **Description:** A documented decision on **what** each wave migrates (mail incl. date-range filter, calendar, contacts, OneDrive, SharePoint), **fidelity expectations**, **throttling limits**, and **exclusions** (e.g. Recoverable Items, huge archives). Captured as `Migration Profile – Mail+Drive – Standard`.

• **Why required:** The profile is the reusable contract for every wave; deciding it once keeps waves consistent and makes pre-stage/delta behavior predictable.

• **How to verify:** Confirm the profile choices are written down and approved by the migration and security owners (retention/eDiscovery implications per GR-6).

• **How to implement if missing:** Run a design workshop to set scope, filters, and throttling; record the decisions; then build the profile in CloudM (**console steps deferred to CloudM docs**).

R7. Change window, cutover plan, and rollback

• **Description:** An approved **change window** for each wave's **MX/mail-routing cutover** (GR-3 DNS), a written **cutover runbook**, and a documented **rollback** (revert MX, keep M365 authoritative) should validation fail.

• **Why required:** Cutover changes mail flow — the highest-impact change in the project. Rollback must be pre-agreed, not improvised.

• **How to verify:** Confirm the change ticket, window, approvers, and rollback steps exist for the wave.

• **How to implement if missing:** Draft the cutover runbook and rollback, route through change control (GR-5), and get sign-off before the wave's cutover.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

Execute the migration following the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback. Every artifact is named with the `<Purpose> – <Scope> – <Stage>` convention and reused verbatim in testing, troubleshooting, and the scripts.

> ***CloudM console internals are deferred to CloudM's documentation.*** *Where a step happens *inside* the CloudM Migrate console (creating the connection object, building the profile, launching a run), the* ***procedure and sequencing*** *are given here; the exact screen labels and clicks are CloudM's to document and may change by version —* ***verify for your version.*** *Google Admin console and Entra steps are exact.*

Stage 1 — Assess (read-only discovery)

1.1 Inventory the source. Run `assess-migration-readiness.py` (read-only) against the Google destination directory and a source export to size the migration: user counts, mailbox and OneDrive sizes, SharePoint site inventory, and license coverage. Save the output as the assessment baseline.

1.2 Confirm quotas and endpoints. Confirm the Google project has the required APIs enabled and that both Microsoft and Google API endpoints are reachable from the platform's network (self-hosted: from each migration server). Record the provider throttling ceilings you expect to hit (Graph per-app/per-mailbox limits, Gmail API quotas) — `plan-migration-waves.py` uses these to advise throttling.

1.3 Verify all requirements. Walk R1–R7 and the global GR-1…GR-6. **Gate 1 (Assess):** every requirement verified, assessment baseline captured. Do not proceed until met.

Stage 2 — Design

2.1 Design waves. Group users into waves aligned to departments/sites. Reserve `Migration Wave – Pilot – Wave0` (25–50 low-risk, cooperative users spanning the data types you must prove: heavy mailbox, large OneDrive, Shared-Drive/SharePoint user). Run `plan-migration-waves.py` to produce a sequenced wave plan with pre-stage/delta ordering and throttling guidance.

2.2 Design the profile. Finalize `Migration Profile – Mail+Drive – Standard` per R6: what to migrate, date-range filters, exclusions, fidelity expectations, and throttling limits.

2.3 Design the user list. Generate the mapping CSV with `build-migration-userlist.py` (dry-run default), reconciling source UPNs to destination Google addresses and file sources to Shared Drives. Align headers to CloudM's import template (**verify for your version**).

Gate 2 (Design): wave plan, profile design, and pilot user list reviewed and approved by the migration + security owners.

Stage 3 — Configure

3.1 Stand up the platform. Confirm `Migration Platform – Enterprise – Prod` is reachable (hosted) or installed/scaled (self-hosted) per R1.

3.2 Build the source connection. In CloudM, create `Source Connection – M365 – Prod` using the Entra app registration from R2 (Application/Directory IDs + secret/cert). *(CloudM console steps deferred — verify for your version.)* Test the connection; it must authenticate and enumerate a sample mailbox.

3.3 Build the destination connection. Create `Destination Connection – Google – Prod` using the service account + DWD from R3. Test; it must authenticate and impersonate a sample destination user. If it fails, re-check the DWD scope list in **admin.google.com** → **Security** → **API controls** → **Domain-wide delegation** (see Troubleshooting).

3.4 Build the profile. Create `Migration Profile – Mail+Drive – Standard` from the R6 design.

3.5 Import the pilot user list. Import the `Migration Wave – Pilot – Wave0` CSV into a run bound to the profile.

Gate 3 (Configure): both connections test green; profile and pilot list imported without errors.

Stage 4 — Pilot (`Migration Wave – Pilot – Wave0`)

Run the full pass sequence on the pilot cohort. **This proves the design before any production wave.**

4.1 Scan (read-only). Launch a **Scan** pass for Wave0. It discovers item counts/sizes per user without writing. Record the scan totals as the expected baseline for validation.

4.2 Pre-stage. Launch `Pre-stage Run – Pilot – Wave0` to bulk-copy historical mail and files into Google while pilot users still work in M365. Monitor throughput; if you hit provider throttling, lower concurrency per `plan-migration-waves.py` guidance (see Troubleshooting).

4.3 Delta + cutover. In the approved change window (R7):

1. Run `Delta Run – Pilot – Wave0` to copy everything changed since pre-stage.
2. Execute the **MX/mail-routing cutover** for the pilot users' domain/routing per the cutover runbook (DNS is high-impact — GR-3).
1. Run a **post-cutover** `Delta Run – Pilot – Wave0` to sweep the final items delivered to M365 during the switch.

Gate 4 (Pilot pass/fail): proceed to validation only if all pilot users completed with errors triaged (not necessarily zero, but every error explained). **Rollback if failed:** revert MX to M365, keep M365 authoritative, retriage, and re-run — no production wave starts until the pilot passes.

Stage 5 — Validate

5.1 Run validation. Execute `Validation Report – Pilot – Wave0` with `validate-migration-report.py`: compare migrated item counts against the scan baseline per user, spot-check Drive/Shared-Drive permissions, and confirm mail is delivering to Google (test message in/out). See `05-testing.md` for the full test matrix and expected results.

5.2 Confirm mail flow. Send a message from an external address to a pilot user and confirm delivery to Gmail; send from the pilot user outbound and confirm SPF/DKIM/DMARC pass.

Gate 5 (Validate): item-count variance within the agreed tolerance, permissions intact, mail flowing both ways, users confirm access. Only now is the profile/connection design blessed for production.

Stage 6 — Expand (production waves)

For each production wave (e.g. `Migration Wave – Finance – Wave3`), repeat the proven sequence:

1. Import the wave's user list into a run on `Migration Profile – Mail+Drive – Standard`.
2. **Scan** → record baseline.
3. `Pre-stage Run – <Dept> – <Wave>` well ahead of the change window.
4. In the change window: `Delta Run` → **MX cutover** (per runbook, rollback ready) → post-cutover `Delta Run`.
1. `Validation Report – <Dept> – <Wave>` — must pass the Gate 5 criteria before the wave is closed.

Sequence waves per `plan-migration-waves.py` so concurrent pre-stage/delta load stays under provider throttling ceilings. Run `check-migration-health.py` between passes to watch error rates, throttling signals, and destination account/license health, and to catch drift before it compounds.

Per-wave rollback (production): if a wave fails validation, revert its MX/routing to M365, keep M365 authoritative for that cohort, triage errors, and re-run the delta+cutover in the next window. Because waves are cohort-scoped, a failure is contained.

Testing & Validation

Each wave must pass these validations before it is closed. Run them at the pilot (`Migration Wave – Pilot – Wave0`) first, then for every production wave. Where a test is scripted, the tool is named. Record results in the wave's `Validation Report – <Scope> – <Stage>`.

> *Item-count variance is measured against the **Scan baseline** captured before the pre-stage pass. Agree a tolerance up front (e.g. ≤0.5% per user, with any excluded item types — Recoverable Items, filtered date ranges — subtracted from the expected count so the comparison is apples-to-apples).*

T1. Connection authentication

- **Procedure:** In CloudM, test `Source Connection – M365 – Prod` and `Destination Connection – Google – Prod`.
- **Expected PASS:** both authenticate; source enumerates a sample mailbox; destination impersonates a sample destination user.
- **FAIL:** auth error or impersonation denied → see Troubleshooting (app-consent / DWD scope mismatch).

T2. Scan baseline captured

- **Procedure:** Confirm the Scan pass completed for every user in the wave and totals were recorded.
- **Expected PASS:** per-user item counts and sizes recorded for mail, OneDrive, and SharePoint in scope; zero users failed to scan.
- **FAIL:** any user missing a scan total → resolve access for that user before pre-stage.

T3. Item-count reconciliation (mail + files)

- **Procedure:** Run `validate-migration-report.py` to compare destination item counts (Gmail messages via Gmail API; Drive files via Drive API) against the Scan baseline, per user, after the final post-cutover delta.
- **Expected PASS:** per-user variance within the agreed tolerance for every user in the wave; variances explained by known exclusions.

• **FAIL:** any user over tolerance → triage in Troubleshooting (incomplete items / throttling truncation), re-run delta, re-validate.

T4. Drive & Shared Drive permissions

• **Procedure:** With `validate-migration-report.py --check-permissions`, spot-check a sample of migrated Drive items and Shared Drives: sharing/membership matches the intended mapping.

• **Expected PASS:** sampled items have the expected members/roles; no unexpected public/"anyone" sharing was introduced.

• **FAIL:** permission drift or over-sharing → correct sharing, review the profile's permission handling (**verify CloudM permission-mapping behavior for your version**).

T5. Mail flow after cutover

• **Procedure:** Send an inbound test (external → migrated user) and an outbound test (migrated user → external) after the MX cutover.

• **Expected PASS:** inbound lands in Gmail; outbound passes **SPF, DKIM, and DMARC** (check the received-message authentication results).

• **FAIL:** non-delivery or auth failure → check MX/DNS records and SPF/DKIM/DMARC alignment for the domain (high-impact — treat under change control).

T6. Calendar & contacts fidelity

• **Procedure:** Spot-check a sample of migrated calendars and contacts against the source.

• **Expected PASS:** future meetings, recurrence, and key contacts present; known-reduced-fidelity items match CloudM's documented fidelity matrix.

• **FAIL:** missing future meetings → re-run with calendar in scope; document any vendor-known fidelity gaps.

T7. Error triage completeness

• **Procedure:** Review the wave's per-user error report; run `check-migration-health.py` for the aggregate error/throttling picture.

• **Expected PASS:** every error is either resolved or explicitly accepted with a reason; no unexplained failures.

• **FAIL:** unexplained errors remain → do not close the wave; resolve or formally accept each.

T8. User acceptance (pilot)

- **Procedure:** Pilot users confirm they can access mail, calendar, and files in Google and that recent items are present.
- **Expected PASS:** pilot users sign off; no blocking access issues.
- **FAIL:** blocking issue → invoke pilot rollback (revert MX, M365 authoritative), fix, re-run.

Wave close gate

A wave is **closed** only when **T1–T7 PASS** (and **T8** for the pilot). Any FAIL routes to Troubleshooting, a corrective delta re-run, and re-validation — or, for a cutover-level failure, the documented rollback.

Troubleshooting

Common failure modes when executing an M365→Google migration with CloudM Migrate, and how to resolve them. Named artifacts (`Source Connection – M365 – Prod`, `Destination Connection – Google – Prod`, `Migration Profile – Mail+Drive – Standard`, `Migration Wave – <Scope> – <Stage>`) refer back to Implementation and Testing.

> ***CloudM console specifics are deferred to CloudM's documentation.*** *Where a fix lives inside the CloudM console, the *symptom and direction* are given here; exact clicks and version behavior are CloudM's to document —* ***verify for your version.***

Connections & authentication

`Destination Connection – Google – Prod` *fails to authenticate / impersonate*

- **Symptom:** Connection test fails, or per-user writes fail with `unauthorized_client` / impersonation denied.
- **Cause:** Domain-wide delegation (DWD) scope mismatch or missing API enablement.
- **Fix:** In **admin.google.com → Security → Access and data control → API controls → Domain-wide delegation**, confirm the service account** Client ID **is present and its scope list** exactly **matches CloudM's Google connector requirements (a single missing scope fails impersonation). Confirm the Cloud project has Gmail, Drive, Admin SDK, Calendar, and People APIs** enabled**. Re-test. `validate-migration-report.py` uses the same DWD path — if it errors on `with_subject`, the grant is the likely culprit.

`Source Connection – M365 – Prod` *fails*

- **Symptom:** Source test fails to authenticate or cannot enumerate mailboxes.
- **Cause:** Entra app registration missing admin consent, missing permissions, or an expired secret/certificate.
- **Fix:** In **Entra admin center** → **App registrations** → your app → **API permissions**, confirm the

permissions CloudM's M365 connector requires are present and show **admin consent granted** (green check). Confirm the client secret/certificate has not expired; rotate if needed and update the CloudM connection (never store the secret in the repo).

Throttling & API limits

Throughput collapses / many 429 / backoff during pre-stage or delta

- **Symptom:** Passes slow dramatically; run log shows HTTP 429, "rate limit", "too many requests".
- **Cause:** Microsoft Graph/Exchange or Gmail/Drive provider throttling — expected under high concurrency.
- **Fix: Lower concurrency**, don't retry blindly. Start from the ceiling `plan-migration-waves.py` recommends and **halve it** on repeated throttling. Keep total simultaneous user migrations across **all** running waves under the ceiling. Shift pre-stage to off-hours. Run `check-migration-health.py --run-log <export>` to quantify throttling signals before re-running.

One user or mailbox is disproportionately slow

- **Symptom:** A single very large mailbox/OneDrive stalls a wave.
- **Cause:** Per-mailbox provider limits and item volume.
- **Fix:** Pre-stage large mailboxes **early and separately** (flagged by `assess-migration-readiness.py --large-mailbox-gb`); consider date-range filtering in the profile for the deepest archives (record the exclusion so validation tolerance accounts for it).

Item counts & delta issues

Item-count variance over tolerance in `Validation Report – <Scope> – <Stage>`

- **Symptom:** `validate-migration-report.py` reports a user FAIL — destination counts below the scan baseline beyond tolerance.
- **Cause:** Incomplete items (throttling truncation, transient errors), or a legitimate exclusion not subtracted from the expected count.
- **Fix:** First confirm the gap isn't an **agreed exclusion** (Recoverable Items, filtered date range) — subtract those from expected. If it's real, **re-run a Delta** for that user, then re-validate. Persistent per-item failures: review the run-log error rows and CloudM's documented handling for those item types (**verify for your version**).

Items delivered after cutover are missing

- **Symptom:** Mail that arrived in M365 during the MX switch isn't in Google.
- **Cause:** No post-cutover delta was run.
- **Fix:** Always run a **post-cutover** `Delta Run – <Scope> – <Stage>` to sweep items delivered to M365 during the switch. Make it a required step in the cutover runbook.

Permissions & sharing

Over-sharing / unexpected "anyone" links flagged

- **Symptom:** `validate-migration-report.py --check-permissions` reports `over-shared` items.
- **Cause:** Source sharing translated to broad Google sharing, or profile permission-mapping settings.
- **Fix:** Correct the sharing on the affected items; review the profile's permission-handling options

and re-run a permission spot-check (**verify CloudM permission-mapping behavior for your version**).

Shared Drive content lands in the wrong place or with wrong members

- **Symptom:** SharePoint/team content in an unexpected Shared Drive or with wrong membership.
- **Cause:** Incorrect file source→Shared Drive mapping in the user list.
- **Fix:** Fix the mapping (regenerate with `build-migration-userlist.py`, review, re-import),

re-migrate the affected content, re-validate T4.

Mail flow after cutover

Mail not delivering to Gmail, or outbound failing SPF/DKIM/DMARC

- **Symptom:** T5 fails — inbound not arriving or outbound authentication failing.
- **Cause:** MX/DNS not fully switched, or SPF/DKIM/DMARC misaligned for Google.
- **Fix:** Verify the domain's **MX** points to Google and that **SPF**, **DKIM** (Google-published

key), and **DMARC** are aligned. Treat DNS changes under change control. If a wave's mail flow can't be made healthy, invoke the documented **rollback** (revert MX, keep M365 authoritative) and re-cut in the next window.

Destination accounts

Users missing or suspended at migration time

- **Symptom:** `check-migration-health.py` reports MISSING or INACTIVE destination accounts; per-user

writes fail.

- **Cause:** Provisioning (GCDS/SCIM) lag, unlicensed, or suspended accounts.
- **Fix:** Provision/license/unsuspend the affected accounts from the identity source of truth before

re-running the wave's delta. Re-run the health check to confirm HEALTHY before proceeding.

Best Practices

Durable practices for executing M365→Google migrations with CloudM Migrate reliably at scale.

Sequencing & waves

- **Always pilot first.** Prove `Migration Profile – Mail+Drive – Standard` and both connections on

`Migration Wave – Pilot – Wave0` before any production wave. Choose pilot users who span the hardest data types (huge mailbox, large OneDrive, Shared-Drive/SharePoint heavy).

- **Pre-stage everything you can.** Move the historical bulk of bytes days before the change window so

the cutover-window delta is small and fast. Cutover risk scales with how much data crosses during the window — keep it minimal.

- **One profile, many waves.** Reuse a single approved profile across waves so behavior is consistent

and predictable; change the profile only through review.

- **Cohort waves to departments/sites.** Keeps communications, support, and rollback contained to a

known group.

- **Sequence waves under the throttling ceiling.** Don't run so many concurrent pre-stage/delta passes

that you saturate Microsoft Graph or Gmail API quotas — use `plan-migration-waves.py` guidance.

Throttling & throughput

- **Expect provider throttling, design for it.** Microsoft (Graph/Exchange) and Google (Gmail/Drive)

both throttle; treat 429/backoff as normal signal, not failure. Tune concurrency down when throttled rather than retrying blindly.

- **Scale self-hosted horizontally** if throughput is the bottleneck — more migration servers/workers,

not just bigger ones (**verify scaling model for your CloudM version**).

- **Run pre-stage off-hours** where provider quotas are less contended; reserve the change window for

the small final delta.

Validation & data integrity

- **Reconcile against the Scan baseline, not against a guess.** Capture per-user scan totals before

pre-stage and compare final counts to them.

- **Agree a variance tolerance up front** and subtract known exclusions (Recoverable Items, filtered

date ranges) so the comparison is honest.

- **Always run a post-cutover delta.** Items delivered to M365 during the MX switch must be swept.

- **Check permissions and over-sharing explicitly.** Confirm migrated Drive/Shared-Drive sharing

matches intent and that no unintended "anyone" links were created.

Change control & safety

- **Treat MX/DNS cutover as the highest-impact change.** Stage it under change control with a written

runbook and a pre-agreed rollback (revert MX, M365 stays authoritative).

• **Keep the source read-only in effect until validated.** Don't decommission M365 mailboxes/data until the wave passes its validation gate — the source is your rollback.

• **Verify SPF/DKIM/DMARC before and after** each cutover; misaligned auth silently damages deliverability.

Security & credentials

• **Never put secrets in code or the repo.** Entra client secrets/certs and the Google service-account JSON key live only in the CloudM platform's secured credential store / your secrets manager. The included scripts use OAuth/ADC and read no secrets from source.

• **Least privilege on both sides.** Grant only the Graph permissions and DWD scopes CloudM's connectors require (**verify the exact lists for your version**); remove them after the project.

• **Rotate and revoke on completion.** Retire the app registration secret, the service-account key, and the DWD grant when the migration closes.

Operations

• **Watch health between passes.** Run `check-migration-health.py` to catch rising error/throttling rates and destination license/account issues before they compound across waves.

• **Document per-wave outcomes.** Keep each `Validation Report – <Scope> – <Stage>` as the audit record of what moved and how it was verified.

• **Defer CloudM specifics to CloudM.** For console behavior, licensing, fidelity matrices, and version-specific features, use CloudM's own documentation and support — this playbook governs procedure and sequencing, not the tool's internals.

Deliverables Summary

Everything included in this solution, on one page.

Documentation

Document	What it covers
Solution Overview	The CloudM Migrate model (platform, connections, profiles, scan/pre-stage/delta), where it fits vs native + other tools, the wave approach, and the deployment model.

Document	What it covers
Requirements Guide	Solution-specific prerequisites R1–R7 (CloudM platform, M365 source app registration, Google service account + DWD, destination users/Shared Drives, user-list mapping, profile design, cutover/rollback) on top of the shared Google global requirements.
Implementation Guide	Exact ordered steps: Assess → Design → Configure → Pilot → Validate → Expand, with named artifacts, pass/fail gates, and rollback. CloudM console internals deferred to CloudM docs.
Testing & Validation	T1–T8 validation procedures with expected pass/fail, including item-count reconciliation, permissions, and mail-flow checks.
Troubleshooting	Common failure modes — throttling/API limits, app-consent/DWD permission errors, incomplete/delta items, mail-flow issues — with resolutions.
Best Practices	Durable practices for reliable sequencing, throttling, validation, change control, and credential hygiene.
Executive Summary	Crisp outcome/approach/why-low-risk summary for decision-makers.

Python tools (5)

All tools are **safe-by-default** (read-only or `--dry-run` unless `--apply`), use **OAuth / ADC** (no hardcoded credentials or keys), and target the **Google Workspace Admin SDK / APIs** (and read the Microsoft side only where noted). Requires Python 3.10+; where Google client libraries are used: `pip install google-api-python-client google-auth`.

Script	What it does
`assess-migration-readiness.py`	Read-only discovery/assessment. Sizes the migration from the Google destination directory (user/license coverage) and a source inventory export (mailbox/OneDrive/SharePoint sizes), and flags gaps. Produces the assessment baseline.
`build-migration-userlist.py`	Builds the source→destination **user-list CSV** that CloudM consumes: reconciles a source user export against the Google directory, maps file sources to Shared Drives. Dry-run by default; `--apply` writes the CSV. Headers align to CloudM's import template (verify for your version).

Script	What it does
`plan-migration-waves.py`	**Wave planner + throttling guidance.** Groups users into sequenced waves (pilot first), orders pre-stage/delta passes, and computes concurrency guidance to stay under Microsoft Graph / Google API throttling ceilings. Read-only planning output.
`validate-migration-report.py`	**Post-migration validation report generator.** Compares destination Gmail/Drive item counts against the scan baseline per user, spot-checks Drive/Shared-Drive permissions (`--check-permissions`), and reports pass/fail per the agreed tolerance. Read-only.
`check-migration-health.py`	**Policy/health/throttling check.** Between passes, watches destination account/license health and surfaces throttling/error signals so drift is caught before it compounds across waves. Read-only.

Packaging

- `CLOUDM-Solution.pdf` — compiled solution document (all sections above).
- `CLOUDM-Scripts.zip` — the five Python tools.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

assess-migration-readiness.py

build-migration-userlist.py

check-migration-health.py

plan-migration-waves.py

validate-migration-report.py